

Final Report – AI Agent System Implementation

Applied System Software (ASD)

Student Name: Herath Mudiyanse Udesha Uditha Kumara Herath

Student ID: 221AMB073

Final System Description and Goal

The system developed is an AI agent written in Python, which uses Google Gemini API for generating responses to user queries and performing tasks that use built-in tools. The system is agent based and is a command-line interface (CLI) application.

The purpose of the system is to showcase the connection between Natural Language Processing and structured tools and how it enables modern AI agents to resolve real-world challenges in a practical manner. The system is modular, extensible and maintainable.

Programming Concepts and Their Usage

Applied important software engineering and programming concepts:

1. Object-Oriented Programming (OOP)

The system is structured using classes such as Agent, Tool Registry and individual tool classes.

2. Modular Design

Each component (agent, tools, memory, registry) is separated into independent modules.

3. Separation of Concerns

Each class has a clear responsibility (e.g., tools execute logic, agent manages flow).

4. Tool Registry Pattern

Tools are dynamically registered and accessed, allowing easy extension without modifying core logic.

5. ReAct Pattern (Reason → Act → Observe):

The agent processes user input, decides on actions, executes tools and returns results.

6. Error Handling

The system safely handles invalid inputs, missing files, and incorrect tool usage.

7. API Integration

The Gemini API is used for reasoning and decision-making.

Tools and Their Roles

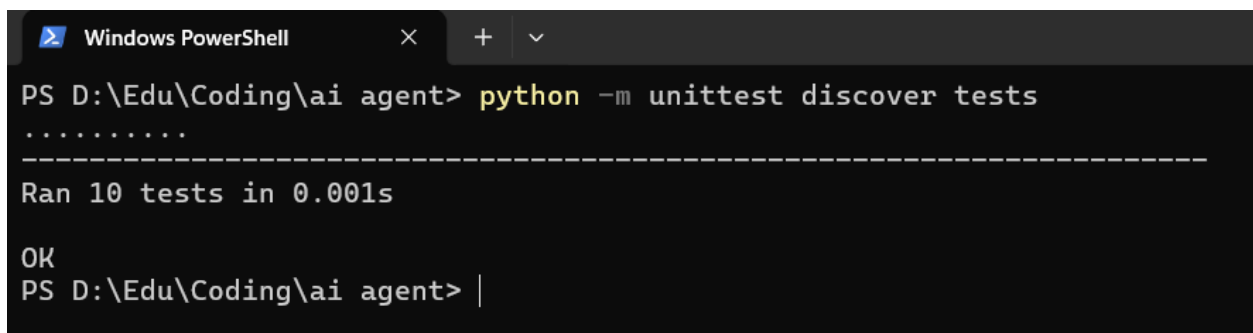
1. Calculator Tool: Performs arithmetic operations based on user input.
2. File Reader Tool: Reads and returns content from text files.
3. Time Tool: Provides the current system time.
4. Translator Tool: Translates text between languages.

Each tool is implemented as a separate class and follows a consistent interface, making the system scalable.

Testing Results and Conclusions

Testing was performed using Python's unittest framework.

Results:



```
Windows PowerShell
PS D:\Edu\Coding\ai agent> python -m unittest discover tests
.....
-----
Ran 10 tests in 0.001s

OK
PS D:\Edu\Coding\ai agent> |
```

Total tests: 10

Passed: 10

Failed: 0

Conclusion:

All system components function correctly under both normal and edge case scenarios. Error handling mechanisms successfully prevent system crashes and provide meaningful feedback to user.

Deployment Preparation

The system is prepared for execution in a controlled environment.

1. Key elements: requirements.txt for dependency management, modular project structure, CLI-based execution
2. Running the system:

```
pip install -r requirements.txt
```

```
python main.py
```

The system is easy to set up and run on any machine with Python installed.

Deployment Strategy

The proposed deployment strategy is a local CLI application.

1. Justification: simple and lightweight, no need for server infrastructure, suitable for testing and demonstration
2. Future improvements could include: deploying as a web service, integrating with APIs

expanding to multi-agent architecture